Linux网络--IO多路转接层01 从阻塞IO到select事件循环

> ⓘ 系列导航
>
> 系列索引：Linux网络--IO多路转接与Reactor系列总览
> 下一篇：Linux网络--IO多路转接层02 poll原理与select服务器改写

- 从阻塞 I/O 到 select 事件循环
  - 1. 一个线程为什么不能一直阻塞在一个连接上
  - 2. 什么叫“就绪”
    - 2.1 读就绪
    - 2.2 写就绪
    - 2.3 异常就绪
  - 3. select 的接口
    - 3.1 nfds
    - 3.2 三个 fd_set
    - 3.3 timeout
    - 3.4 返回值
  - 4. fd_set 为什么既是输入又是输出
    - 4.1 用 bit 状态理解一次 select
  - 5. select 的工作过程
  - 6. 板书中的 select 逻辑
  - 7. 用类封装 select
    - 7.1 Selector 的数据
    - 7.2 添加连接
    - 7.3 删除连接
    - 7.4 等待就绪事件
  - 8. select TCP 服务器的事件循环
    - 8.1 监听 fd 分支
    - 8.2 普通连接分支
  - 9. select 的常见坑
  - 10. select 的边界

从阻塞 I/O 到 select 事件循环

1. 一个线程为什么不能一直阻塞在一个连接上

最直接的服务器模型是“一个连接对应一个线程”。主线程调用 `accept` 得到新连接后创建线程，工作线程在该连接上阻塞调用 `recv`。模型容易理解，但连接数增加后会遇到三类成本：

1. 每个线程都需要独立的栈和调度资源；
2. 大量线程大部分时间都在等待，线程上下文切换却没有产生有效计算；
3. 在线程之间共享连接状态、做管理和回收会引入同步问题。

网络 I/O 可以拆成两个阶段：

```
等待内核缓冲区可读或可写   ->   在用户态和内核态之间拷贝数据
        “等”                          “拷贝”
```

对于大量长连接，最浪费的是“等”。如果能让一个线程同时等待很多 fd，那么只有真正就绪的连接才会进入后续处理，这就形成了 I/O 多路转接。

`select` 不替程序完成 `recv` 或 `send`。它在等待阶段回答一个问题：

> 在这组 fd 中，现在哪些已经可以读、可以写，或者发生了异常？

2. 什么叫“就绪”

就绪不是“数据已经全部到达”，也不是“请求已经完整”。它只是当前这一次 I/O 操作可以取得进展。

2.1 读就绪

常见读就绪条件包括：

- socket 接收缓冲区中的可读字节数达到低水位标记 `SO_RCVLOWAT`；
- TCP 对端正常关闭写方向，此时该 socket 可读，`recv` 会返回 `0`；
- 监听 socket 的已完成连接队列中存在待处理连接，可以调用 `accept`；
- socket 上存在待处理错误。

因此，“监听 fd 可读”通常意味着有新连接可以接收；“普通连接可读”可能意味着收到数据、对端关闭或发生错误，必须在读取后检查具体返回值。

2.2 写就绪

写就绪通常表示发送缓冲区中仍有足够空间，可以继续写入。常见情况包括：

- 发送缓冲区中的可写空间达到 `SO_SNDLOWAT`；
- 非阻塞 `connect` 已经完成或已经失败；
- 连接状态发生变化，需要读取 socket 错误。

在刚建立连接时，发送缓冲区通常是空的，所以写事件往往默认成立。服务端不能因此一直关心写事件，否则事件循环会反复发现“可写”，却没有真正要发送的数据。

2.3 异常就绪

异常事件更少见，通常与带外数据、协议异常或连接状态有关。日常服务器最常用的是读、写以及读过程中的错误和连接关闭。

3. select 的接口

```
#include <sys/select.h>
int select(
    int nfds,
    fd_set *readfds,
    fd_set *writefds,
    fd_set *exceptfds,
    struct timeval *timeout
);
```

五个参数分别控制监视范围、三类事件和等待方式。

3.1 nfds

`nfds` 是“需要检查的最大 fd 加一”。

假设当前关心 `3、5、8` 三个 fd，那么最大 fd 是 `8`，应传入 `9`。内核会在 `[0, nfds)` 范围内检查集合，因此这个参数少一就会漏掉最大 fd，传得过大会增加无意义的检查。

3.2 三个 fd_set

`fd_set` 是位图，不是 fd 数组。某一个 bit 的位置表示 fd 编号，bit 的值表示该 fd 是否属于当前关心的集合。

标准环境常见：

```
FD_SETSIZE == 1024
```

它表示这一版 `fd_set` 能表达的编号范围。它不等于“整个进程最多只能打开 1024 个文件”，只是 `select` 的集合接口通常只能安全覆盖这么多编号。

四个操作宏分别完成：

```
FD_ZERO(&set);        // 清空全部 bit
FD_SET(fd, &set);     // 把 fd 对应的 bit 置 1
FD_CLR(fd, &set);     // 把 fd 对应的 bit 置 0
FD_ISSET(fd, &set);   // 判断 fd 对应的 bit 是否为 1
```

3.3 timeout

`struct timeval` 包含秒和微秒：

```
struct timeval {
    time_t      tv_sec;
    suseconds_t tv_usec;
};
```

它有三种典型用法：

- `timeout == NULL`：一直阻塞，直到至少一个事件或错误发生；
- `*timeout == {0, 0}`：只检查一次，立即返回；
- `*timeout` 为具体时长：最多等待该时长，超时返回。

Linux 的 `select` 可能修改传入的 `timeval`，所以需要保留固定超时值时，每一轮都应重新填写。

3.4 返回值

- `ret > 0`：有 `ret` 个 fd 就绪；
- `ret == 0`：等待超时；
- `ret < 0`：失败，需要检查 `errno`。

常见错误：

- `EBADF`：集合中存在无效 fd；
- `EINTR`：等待被信号打断；
- `EINVAL`：参数不合法，例如 `nfds` 为负数。

遇到 `EINTR` 不应当直接把连接当作关闭，通常应重新进入等待。

4. fd_set 为什么既是输入又是输出

调用前：

```
用户 -> 内核
bit 位置：哪个 fd
bit 内容：是否关心该 fd
```

调用后：

```
内核 -> 用户
bit 位置：哪个 fd
bit 内容：该 fd 是否就绪
```

同一块内存承载了两种含义，而且内核会修改它。假设原本监视 `1、2、5`，返回时只有 `1、2` 就绪，那么 `5` 对应的 bit 会被清掉。

因此，事件循环通常需要两份集合：

```
fd_set all_read_fds;   // 用户长期维护的关心集合
fd_set ready_read_fds; // 本次调用交给内核的临时集合

ready_read_fds = all_read_fds;
int ret = select(max_fd + 1, &ready_read_fds, NULL, NULL, NULL);
```

如果不使用临时副本，本轮返回结果就会污染下一轮想要关心的集合。

4.1 用 bit 状态理解一次 select

假设当前关心 fd `0、4、5`，用一组简化的 bit 表示：

```
调用前：fd 5 4 3 2 1 0
        bit 1 1 0 0 0 1
```

调用 `select` 后如果只有 fd `4、5` 就绪：

```
调用后：fd 5 4 3 2 1 0
        bit 1 1 0 0 0 0
```

返回值是 `2`，但程序不能直接假定前两个 fd 是就绪项。返回值只给出数量，具体是谁仍要逐位调用 `FD_ISSET` 检查。

5. select 的工作过程

一次 `select` 可以理解为下面的过程：

```
用户态构造关心集合
        |
        v
复制集合和超时信息到内核
        |
        v
内核在等待队列中登记当前任务
        |
        v
某个 fd 就绪，内核唤醒等待者
        |
        v
内核修订集合，把就绪项写回
        |
        v
应用扫描集合，处理就绪 fd
```

在没有事件时，等待任务会睡眠，不持续占用 CPU。事件到来后，`select` 返回，应用才进行真正的业务处理。

`select` 的成本主要来自以下重复工作：

1. 每轮重新构造关心集合；
2. 每轮把集合从用户态复制到内核；
3. 内核每轮检查集合中的 fd；
4. 返回时把结果复制回用户态；
5. 应用从 `0` 扫描到 `max_fd`，找出就绪项。

当 fd 很少、大多数都很活跃时，这套接口简单而可靠；当 fd 很多、活跃比例很低时，重复复制和全量扫描就会成为负担。

6. 板书中的 select 逻辑

这张板书从 I/O = 等待 + 拷贝开始，依次画出用户态、内核态、fd 就绪、位图回写和用户态扫描。

读图时抓住四条线：

1. **等待线**：`select` 只在事件就绪前阻塞。
2. **数据线**：返回后仍由应用执行 `read/recv`。
3. **位图线**：fd 编号决定 bit 位置，0/1 决定是否关心或是否就绪。
4. **范围线**：内核检查 `[0, nfds)`，因此最大 fd 必须加一。

板书用 `3、5、8` 为例，最后交给 `select` 的范围是 `0..8`。返回后原来的位图只剩就绪位，这正是事件循环必须保留临时副本的原因。

7. 用类封装 select

直接把 `fd_set`、`max_fd` 和 fd 对象散落在业务代码里，会让新增和删除连接非常容易出错。可以先把等待逻辑封装成一个 `Selector`。

7.1 Selector 的数据

```
class Selector {
private:
    fd_set read_fds_;
    int max_fd_;
    std::unordered_map<int, TcpSocket> fd_map_;
};
```

三份数据分别表示：

- 当前关心哪些 fd；
- 当前最大 fd；
- fd 与 socket 对象的映射。

7.2 添加连接

添加一个 fd 时要做四件事：

```
bool Add(const TcpSocket& sock) {
    int fd = sock.GetFd();
    if (fd_map_.find(fd) != fd_map_.end()) {
        return false;
    }

    fd_map_[fd] = sock;
    FD_SET(fd, &read_fds_);
    if (fd > max_fd_) {
        max_fd_ = fd;
    }
    return true;
}
```

必须同时更新集合、映射和 `max_fd_`。如果只设置位图却不更新最大 fd，后面的 `select(max_fd_ + 1, ...)` 可能根本检查不到新连接。

7.3 删除连接

删除连接时：

```
bool Del(const TcpSocket& sock) {
    int fd = sock.GetFd();
    if (fd_map_.erase(fd) == 0) {
        return false;
    }

    FD_CLR(fd, &read_fds_);

    if (fd == max_fd_) {
```

```
        while (max_fd_ >= 0 &&
               !FD_ISSET(max_fd_, &read_fds_)) {
            --max_fd_;
        }
    }
    return true;
}
```

删除最大 fd 后需要重新寻找当前最大 fd，否则扫描范围仍会过大。

7.4 等待就绪事件

```
bool Wait(std::vector<TcpSocket>* output) {
    output->clear();

    fd_set ready = read_fds_;
    int n = select(max_fd_ + 1, &ready, NULL, NULL, NULL);
    if (n < 0) {
        if (errno == EINTR) return true;
        return false;
    }

    for (int fd = 0; fd <= max_fd_; ++fd) {
        if (!FD_ISSET(fd, &ready)) continue;

        auto it = fd_map_.find(fd);
        if (it != fd_map_.end()) {
            output->push_back(it->second);
        }
    }
    return true;
}
```

这里使用 `ready` 临时集合，避免内核修改 `read_fds_`；同时只扫描到 `max_fd_`。

8. select TCP 服务器的事件循环

服务器启动时只有监听 socket。客户端连接后会新增普通 socket，因此同一个事件循环要处理两类 fd。

```
int main_loop() {
    TcpSocket listen_sock;
    listen_sock.Socket();
    listen_sock.Bind(ip, port);
    listen_sock.Listen(backlog);

    Selector selector;
    selector.Add(listen_sock);

    for (;;) {
        std::vector<TcpSocket> ready;
        if (!selector.Wait(&ready)) {
            if (errno == EINTR) continue;
            return -1;
        }

        for (TcpSocket& sock : ready) {
            if (sock.GetFd() == listen_sock.GetFd()) {
                TcpSocket client;
                if (listen_sock.Accept(&client, nullptr, nullptr)) {
                    selector.Add(client);
                }
                continue;
            }

            std::string request;
            if (!sock.Recv(&request)) {
                selector.Del(sock);
                sock.Close();
                continue;
            }

            std::string response;
            handler(request, &response);
            sock.Send(response);
        }
    }
}
```

8.1 监听 fd 分支

监听 socket 的可读不是“收到了业务数据”，而是“有新的连接可以取出”。因此这里调用 `accept`，再把返回的新 socket 加入 `Selector`。

8.2 普通连接分支

普通连接可读后：

1. 调用 `recv`；
2. 成功则交给业务 handler；
3. 返回 `0` 或以错误结束，则从 select 集合删除；
4. 关闭文件描述符；
5. 继续处理下一个就绪连接。

删除和关闭的顺序最好保持“先删除管理关系，再关闭资源”。这样后续代码不会继续把一个已经失效的 fd 当作可管理对象。

9. select 的常见坑

- 把 `nfds` 写成最大 fd，而不是最大 fd 加一；
- 直接让内核修改长期关心集合；
- 调用返回后忘记用 `FD_ISSET` 查询；
- 返回 `n` 后只处理前 `n` 个 fd，误以为返回数组天然有序；
- 删除最大的 fd 后没有重算 `max_fd`；
- 只处理读事件，忽略 socket 错误和关闭；
- 把 `EINTR` 当成致命错误；
- 把 `FD_SETSIZE` 与进程 fd 上限混为一谈；
- 无条件关心写事件，造成“永远可写”的忙循环。

10. select 的边界

`select` 的优点是可移植、模型直观、小规模连接足够使用。它的主要问题不是“位图太小”这一项，而是每轮重复登记、复制和扫描。

poll 保留了 select“由应用传入一组等待对象”的基本模型，但把输入事件和输出事件拆成了两个字段。下一篇从 select 服务器出发，逐段改写成 poll 服务器，并说明哪些问题解决了，哪些问题仍然存在。